(bottom left) k==75, the method using brute-force is initially taking a similar time, but as `n` grows, the curves for the two running times diverge and k-d tree implementation grows more slowly.
(bottom right) k==200, the k-d tree implementation takes half the time of the brute-force approach even for small values of `n`, and it grows much more slowly.

Once again, we used a Python implementation to do the comparison: by now it should go without saying, but these results are obviously only significant for this language and this particular implementation. Nevertheless, they do prove something.

For k-d trees, we used SciPy's implementation[10] provided in module `scipy.spatial`. If you have read this book's earlier chapters, you probably remember one of the golden rules we mentioned: before implementing something yourself from scratch, look if there is something trustworthy already available. In this case, SciPy's implementation is not only likely more reliable and efficient than the version we could write ourselves (since SciPy's code has been already tested and tuned at length), it also implements the `query` method, performing NN search, by returning the index (relative to the order of insertion) of the point returned: that's exactly what we need for our k-means method, and it will save us from storing extra data to get from points to indices.

Figure 12.7 shows the result of profiling for the Python method cluster_points implemented with brute-force search, and k-d tree's nearest neighbor.